PARALLELISING ML – MODELS

Adrian Jackson

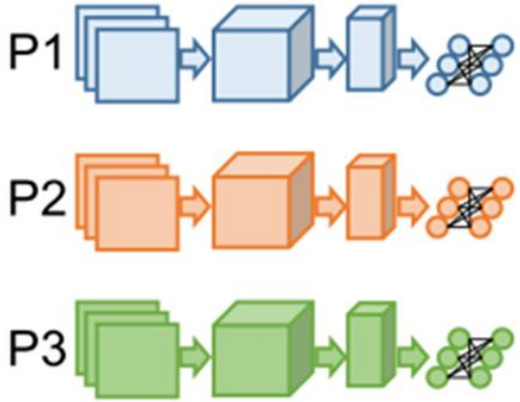
a.jackson@epcc.ed.ac.uk

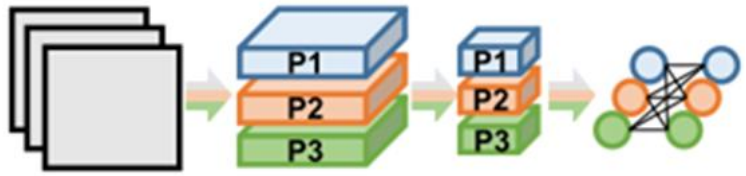
THE UNIVERSITY of EDINBURGH

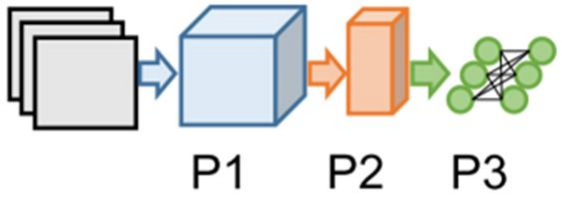
epcc

Multi-resource usage

- Any model or dataset that needs more compute time or memory than a single GPU can provide
  - Need to parallelise some aspect to improve performance
- Standard parallelisation approaches apply
  - Simple parallelisation
    - Inference at scale
  - Advanced task farming
    - Data Parallel: Same model everywhere, different data
  - **Distributed data parallelisation**
    - Model Parallel: Parts of the model split up
    - Domain decomposition (tensor parallelisation)
  - **Distribute function parallelisation**
    - Model Parallel: Parts of the model split up
    - Pipelining

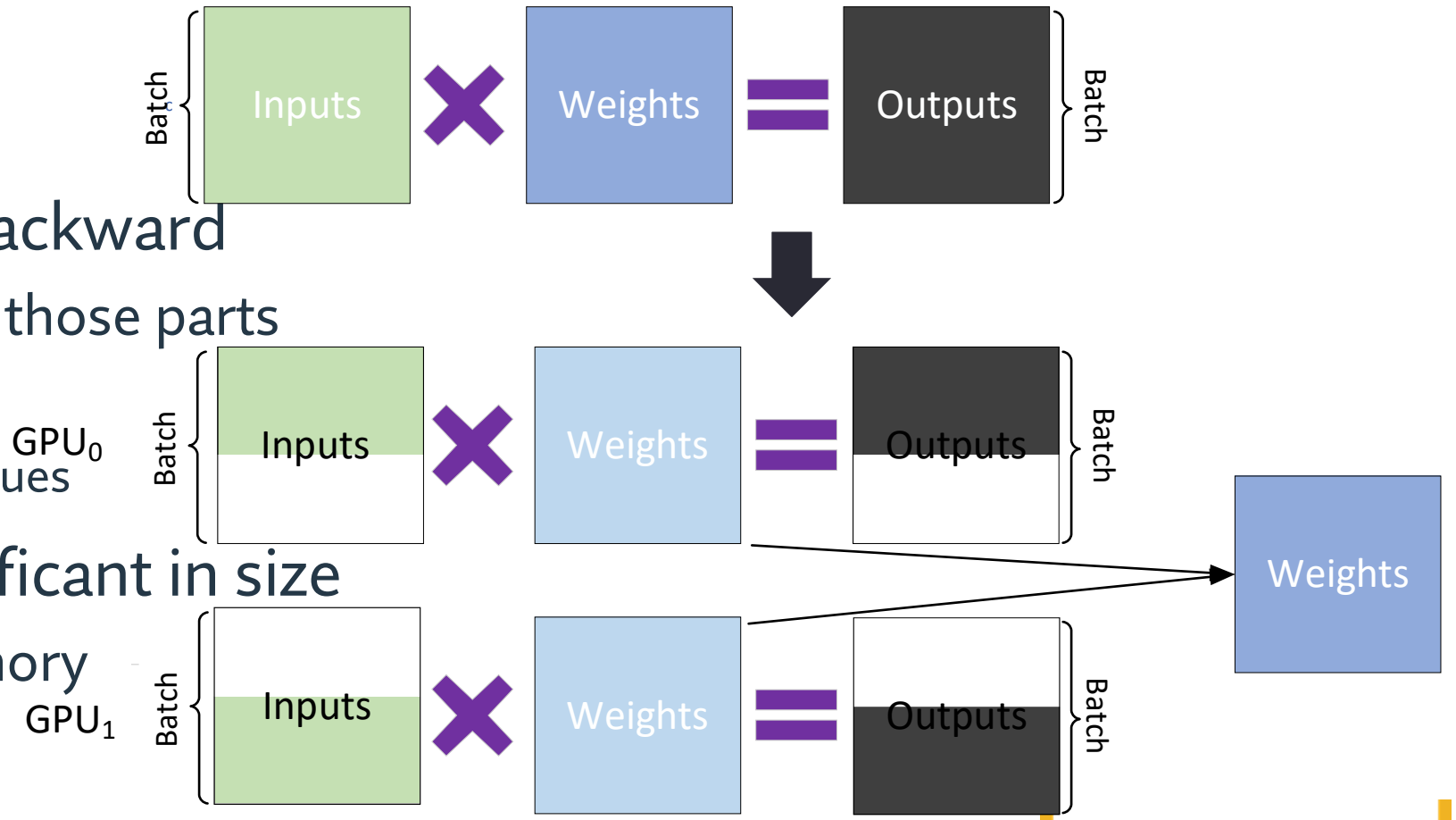






Data Parallel

- Data parallel has a communication cost associated with the weight all reduce
  - Weights size x #GPUs
- Can do forward and backward
  - No communications for those parts
  - Requires full size layers
    - Weights and activation values
- Activation values significant in size
  - Batching increases memory



Model parallelisation

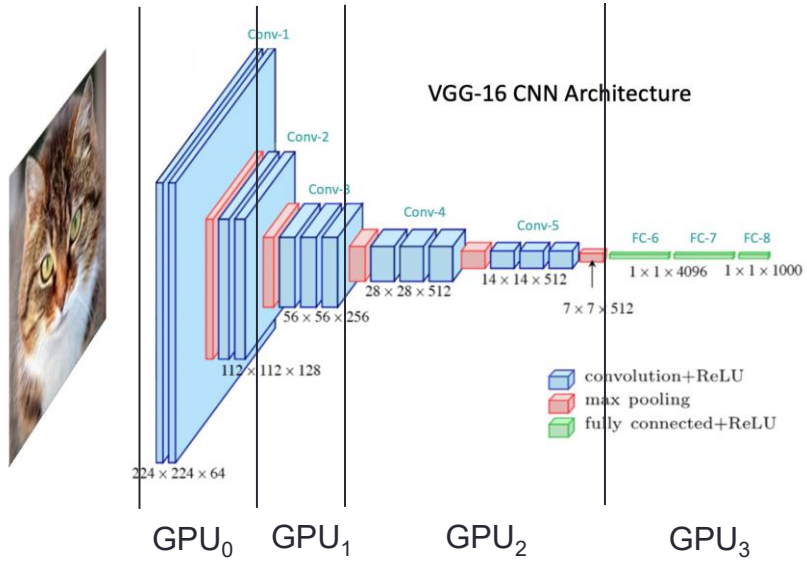
- Need more memory
  - Weights + model doesn't fit in memory
  - Single batch/example input doesn't fit in memory
- Need more compute
  - Single model execution takes too long
- Need different compute
  - Different parts of the model need different hardware (speculative)
- Have to split up the model
  - Distribute some parts of the model to varied resources
  - Different methods for doing this
- Avoid all to all style communications
  - Can overlap communication and computation

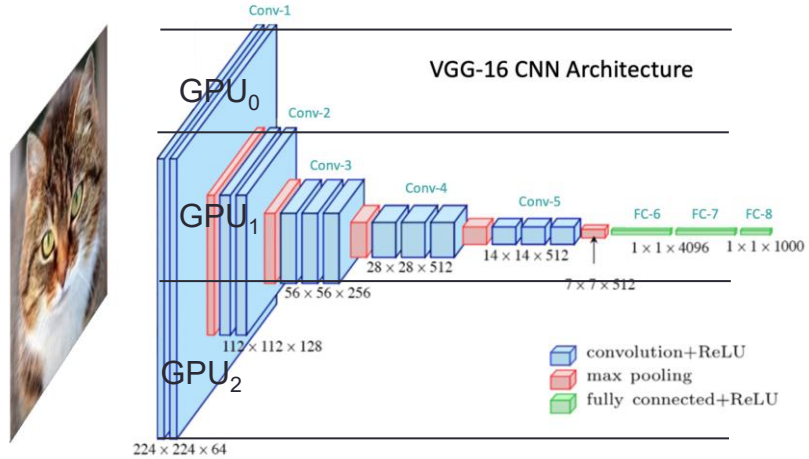
Automatic approaches

- Parameter servers or similar
  - Data parallel approaches but can reduce memory loads depending on how implemented
- Some frameworks are moving to automatically scaling model functionality and data together
  - Zero Redundancy Optimizer (ZeRO): Distribute some parts of the optimiser to reduce memory
- Fully Sharded Data Parallel (FSDP) takes this further
  - More later in the presentation
- Automatic approach does not allow for selection of exact functionality to be distributed
  - Load balance and communication performance issues

Model level parallelisms

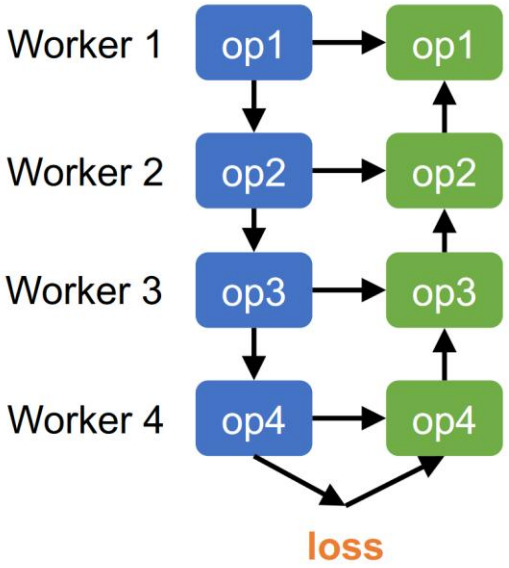
- Pipeline Parallelism (Vertical)
  - Distribute DNN layers to separate GPUs
  - Each layer computed independently
  - Data passed between layers
  - Can have multiple batches in flight at any point of time
- Tensor Parallelism (Horizontal)
  - Split tensor (i.e. weights/layer) up into multiple parts
  - Parts (shards) distributed across GPUs
  - Compute in parallel
  - Communicate at the end of the calculations

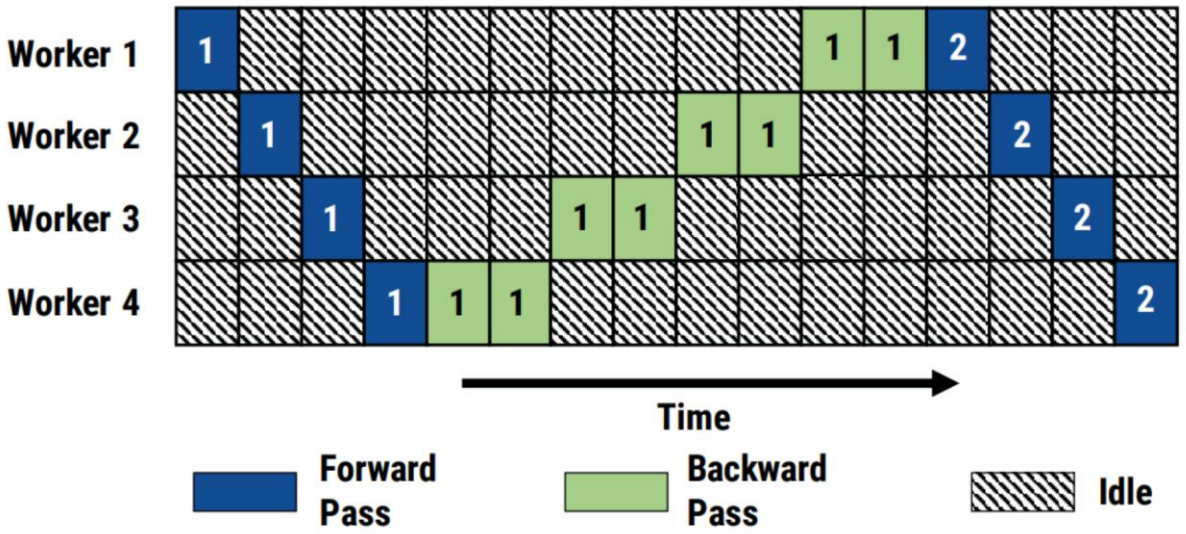




Pipeline parallelism

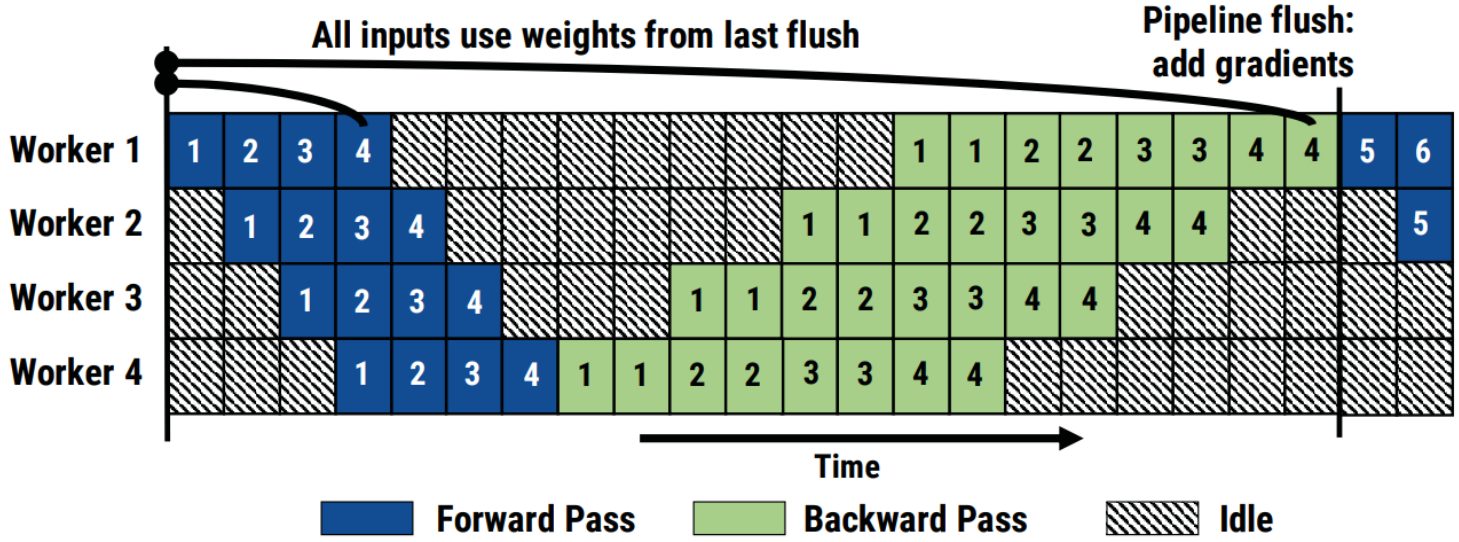
- Divide the model into stages
  - Each stage on a separate resource
  - Communication required between the stages (both forward and backward)
  - Dependencies can cause resource underutilisation





Pipeline parallelisation

- Just like CPU instruction execution, can be optimised with registers and local pipelining
  - Decompose into micro batches to run multiple groups through the pipeline
  - Have to wait at the end of the batch
  - Gpipe paper had a 4x micro-batch to mini-batch with minimal idle time

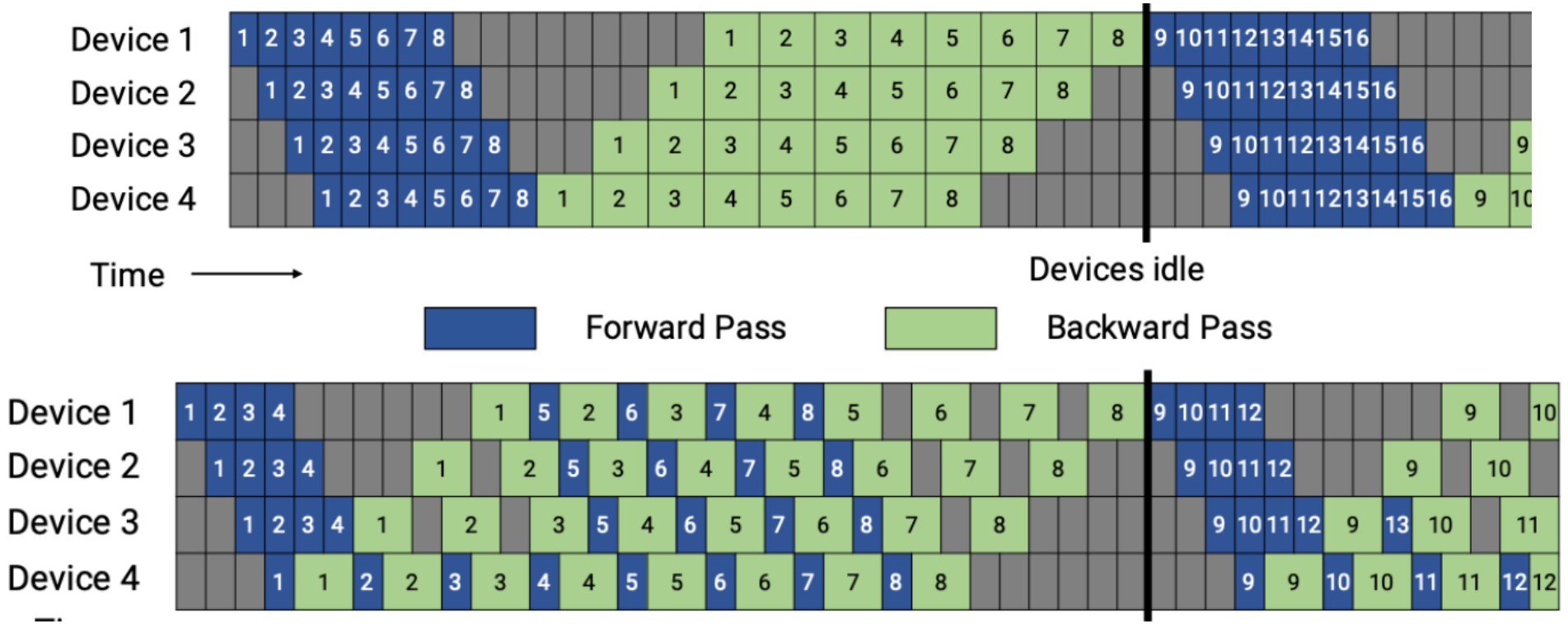


Pipeline parallelism

- Another trade off in batch sizes
  - Micro vs mini batch choices
  - Remember: Large mini-batch can reduce generalisability
  - Small micro-batch and/or small mini-batch reduces GPU utilisation
- Overlap communications and calculations
  - Send forward data for micro batch whilst progressing to next micro batch or to backwards set
- Similar memory issues to general batching
  - Have to hold micro batch activations in memory until backwards pass

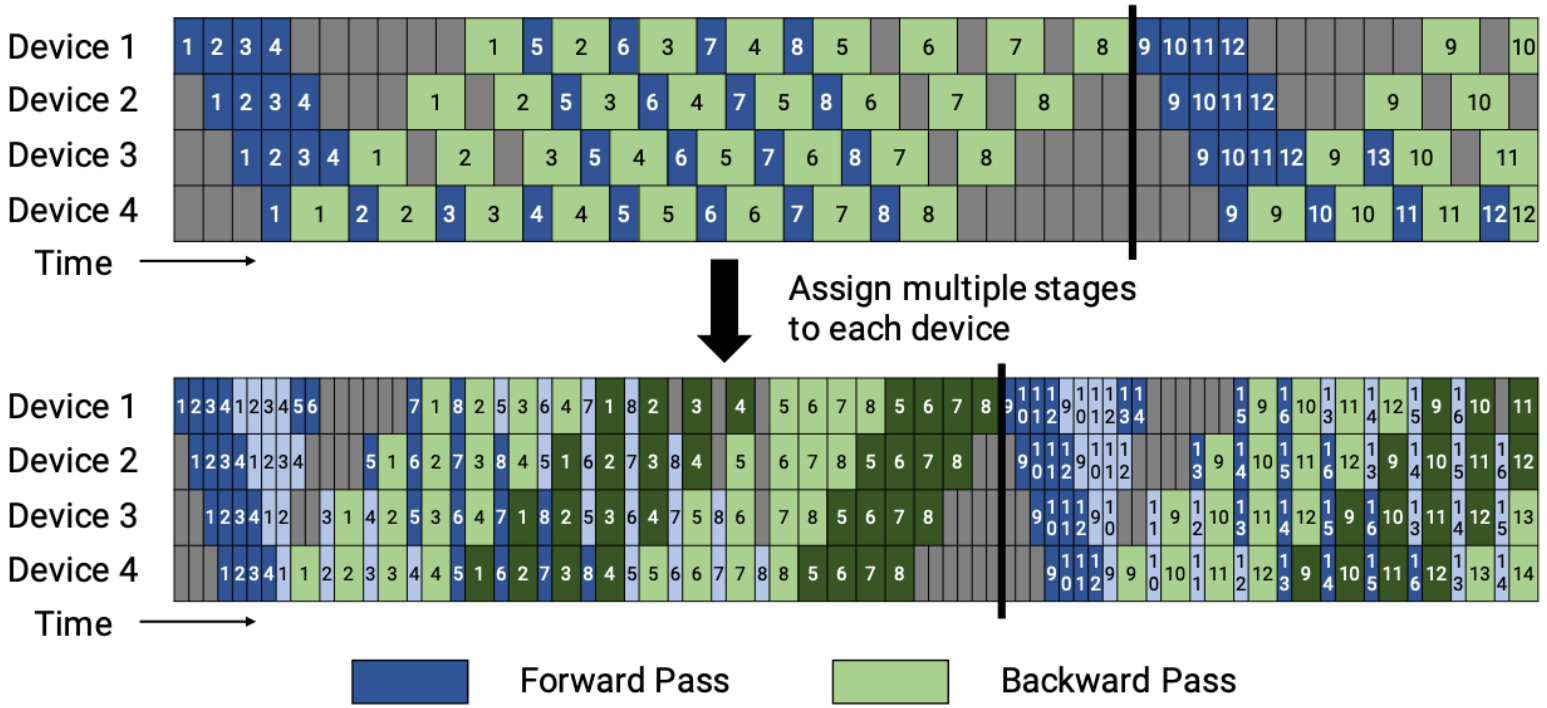
Pipeline parallelism

- Pipeline scheduling can be optimised to improve performance, reduce memory, etc...

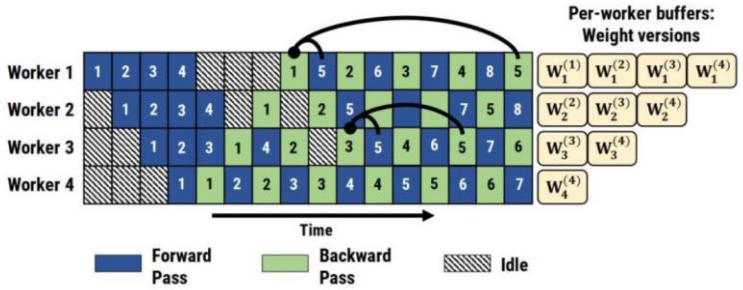


Pipeline parallelism

- Can further decompose to reduce empty stages
  - At the cost of communications
  - Interleave parts of individual micro-batches



Pipeline parallelism



- Performance depends on usage but also load balance
  - Distributing stages may be hard to load balance
  - Classic task decomposition challenges
- Dependencies between forward and backwards pass of layer constrains decomposition/scheduling
- Need to keep track of weights/gradient data if micro-batching or overlapping mini-batching
  - Weight stashing
  - In-flight pipelining optimisation requires this functionality
  - PipeDream paper, run forward and backwards passes alternatively (1F1B) rather than all forward then all backwards
- Re-materalisation
  - Can do activation re-materialisation to reduce memory requirements
  - Only store activations at pipeline boundary, recompute the rest when required.

Implementing pipeline parallelism

- PyTorch has a basic pipeline implementation
  - Based on the original Gpipe approach

`torch.distributed.pipeline.sync.Pipe`

  - Wraps `nn.Sequential` network
  - Developer needs to do the placement of layers and ordering
  - Each layer needs to be on a single device
  - Automatically splits mini-batches into micro-batches (first dimension of input tensor)
  - Only partitions tensors, not other data structures

```
torch.distributed.rpc.init_rpc('worker', rank=0, world_size=1)
fc1 = nn.Linear(16, 8).cuda(0)
fc2 = nn.Linear(8, 4).cuda(1)
model = nn.Sequential(fc1, fc2)
model = Pipe(model, chunks=8)
input = torch.rand(16, 16).cuda(0)
output_results = model(input)
```

Implementing pipeline parallelism

- `Pipe` also supports skip connections
  - Pass data between layers without passing through all layers
  - Uses `@skippable` decorator with `stash` and `pop` functions

```
@skippable(stash=['1to3'])
class Layer1(nn.Module):
    def forward(self, input):
        yield stash('1to3', input)
        return f1(input)
class Layer2(nn.Module):
    def forward(self, input):
        return f2(input)
@skippable(pop=['1to3'])
class Layer3(nn.Module):
    def forward(self, input):
        skip_1to3 = yield pop('1to3')
        return f3(input) + skip_1to3
model = nn.Sequential(Layer1(), Layer2(), Layer3())
```

  - Supported in non-`Pipe` implementations as well

Implementing pipeline parallelisation

- TensorFlow doesn't provide built in pipeline functionality
  - Lower-level TensorFlow gives direct control of layer location and already manages data transfers
  - Can also track where layers are running using `tf.debugging.set_log_device_placement(True)`

```
import tensorflow as tf
# Define a simple model
class MyModel(tf.keras.Model):
    def __init__(self):
        super(MyModel, self).__init__()
        self.layer1 = tf.keras.layers.Dense(128, activation='relu')
        self.layer2 = tf.keras.layers.Dense(64, activation='relu')
        self.layer3 = tf.keras.layers.Dense(64, activation='relu')
        self.layer4 = tf.keras.layers.Dense(10)

    def call(self, inputs, training=False):
        with tf.device('/GPU:0'):
            x = self.layer1(inputs)
            x = self.layer2(x)
        with tf.device('/GPU:1'):
            x = self.layer3(x)
            x = self.layer4(x)
        return x

# Create and compile the model
model = MyModel()
model.compile(optimizer='adam', loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True), metrics=['accuracy'])
# Dummy dataset
import numpy as np
x_train = np.random.random((1024, 20)).astype(np.float32)
y_train = np.random.randint(10, size=(1024, 1)).astype(np.float32)

# Train the model
model.fit(x_train, y_train, epochs=10)
```

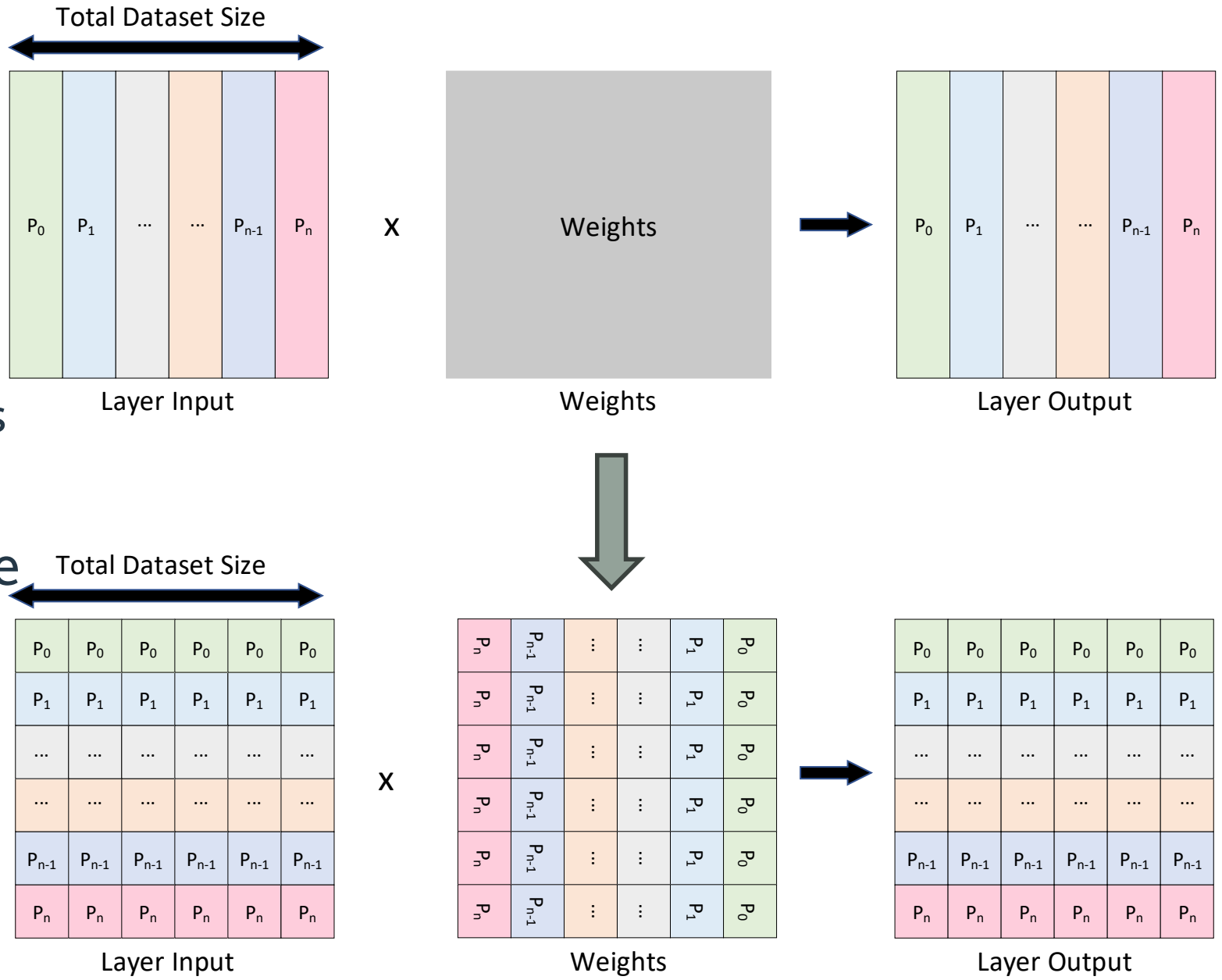
Implemented pipeline parallelism

- TensorFlow communication/sync functionality can be used for explicit data movements
  - Generally the graph functionality will handle this, but if there are more complex requirements (skip connections, recurrences, etc...) may need to be explicit
  - `tf.identity()` will communicate a variable/tensor from one place to another
  - `tf.control_dependencies()` can specify graph requirements for operations to occur

```
stage2_inputs = tf.identity(stage1_outputs)
....
with tf.control_dependencies([op1, op2]):
    op3 = tf.add(x, y)
```

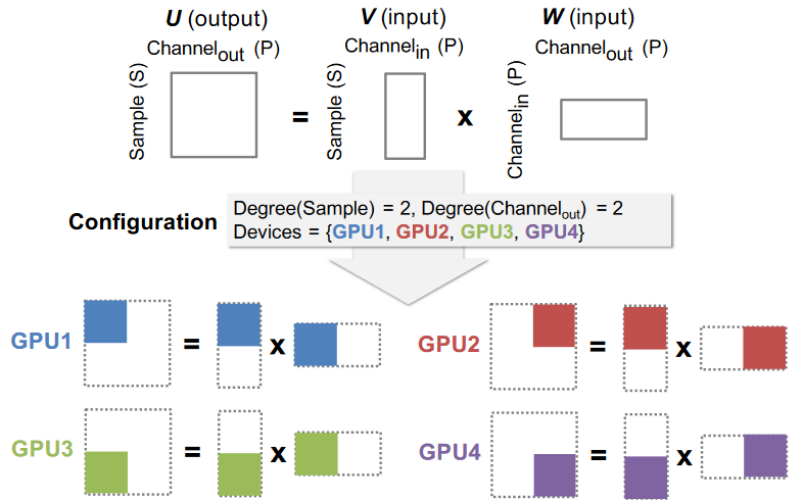
Tensor parallelism

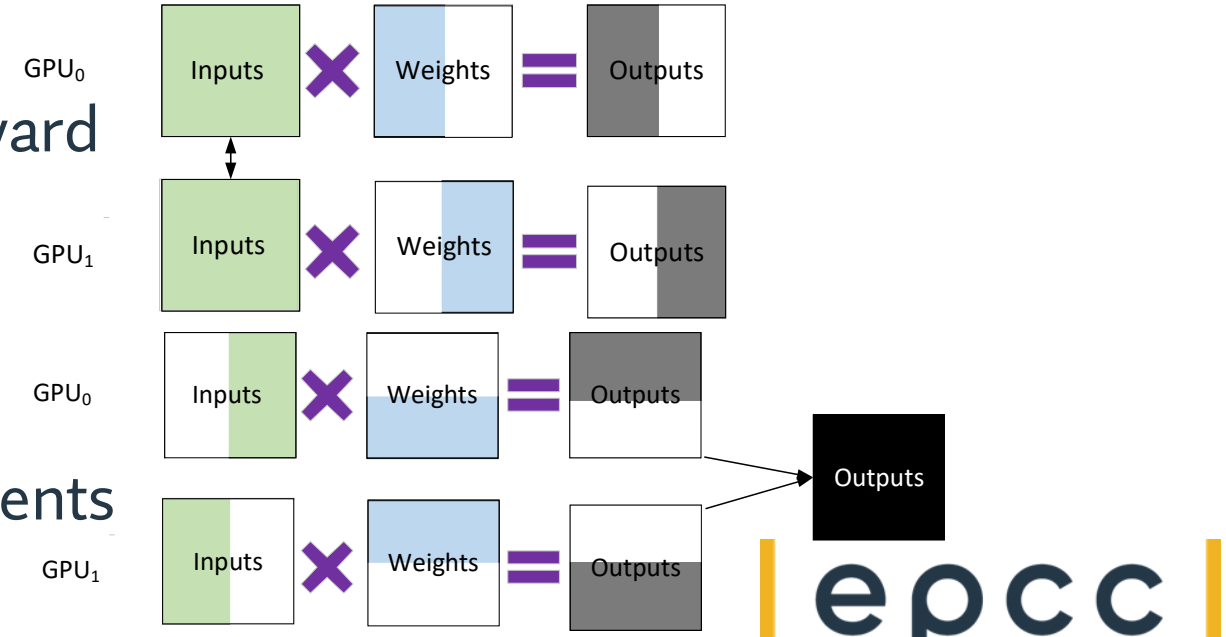
- Shard layers
  - Full size partitioned across GPUs
- Enables reducing the size of weight locally
- Full weights x input may be decomposed anyway
  - Blocked for improved performance



Different tensor parallelism approaches

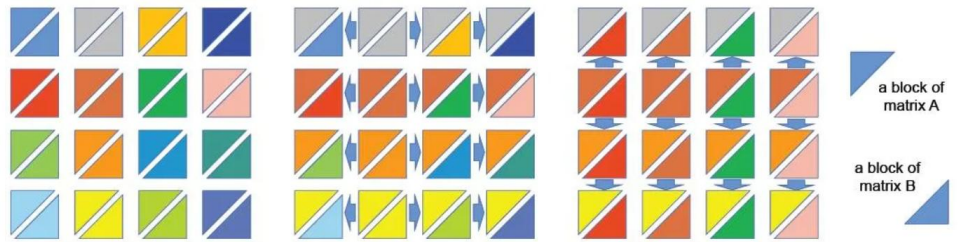
- Varies by how you split the weights/activations
  - Decompose columns or rows of weights
  - Depends on where the size/cost is
- Decomposed output
  - Results of forwards pass are partial solution spaces
  - Full results in each partitioned space
  - Requires comms at forward and backward
- Gathered output
  - Partition both inputs and weights
  - End up with full partial output on all
  - Need to sum these partials to get gradients





Tensor Parallelism

- Overall approach depends on what is expected layer to layer
  - Can decompose all layers in same pattern to defer communications to the end
  - May require more data to be distributed
  - Depends on network structure
- Also can impact the communications required for gradient calculations
- i.e. 1D vs 2D domain decomposition
  - Communications can be reduced for 2D
  - Column/row splitting vs both



An illustration of SUMMA

Implementing tensor parallelisation

- TensorFlow `dtensor` library provides general tensor distribution
- `tf.experimental.dtensor`
  - Uses a `dtensor.Mesh` and `dtensor.Layout` to define how to shard a tensor across resources
  - Mesh defines the device list
  - Layout defines how to shard the Tensor onto the mesh
  - Can think of this list MPI cartesian topologies
- For mesh you specify dimension and sizes

```
mesh_1d = dtensor.create_mesh([('x', 6)], devices=DEVICES)
mesh_2d = dtensor.create_mesh([('x', 3), ('y', 2)], devices=DEVICES)
```

- For layout you specify how to split dimension across the mesh

```
layout = dtensor.Layout([dtensor.UNSHARDED, dtensor.UNSHARDED], mesh_1d)
layout = dtensor.Layout([dtensor.UNSHARDED, 'x'], mesh_1d)
```

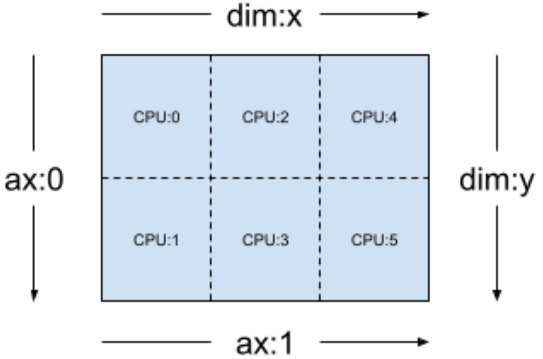
Implementing tensor parallelisation

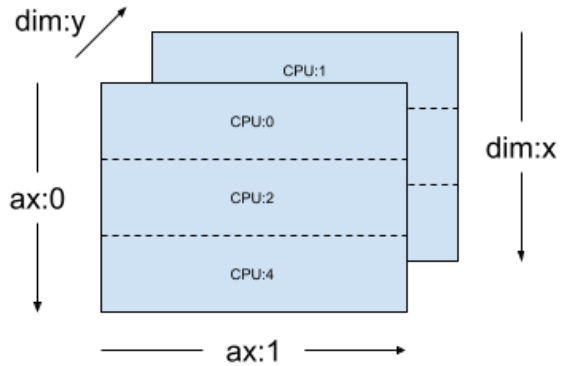
- Can mix and match replication and sharding, i.e.:

```
layout = dtensor.Layout(['y', 'x'], mesh_2d)
layout = dtensor.Layout(["x", dtensor.UNSHARDED], mesh_2d)
```

- Can then create distributed tensors

```
hybrid_sharded_dtensor = dtensor_from_array(
    tf.reshape(tf.range(6), (3, 2)),
    layout=dtensor.Layout(['x', dtensor.UNSHARDED], mesh))
 self.weight = dtensor.DVariable(
        dtensor.call_with_layout(
            random_normal_initializer, weight_layout,
            shape=[input_size, output_size],
            seed=init_seed))
```





Implementing tensor parallelisation

- Then have to use `dtensor` in your layers

```
class MLPStricter(tf.Module):

  def __init__(self, mesh, input_mesh_dim, inner_mesh_dim1, output_mesh_dim):
    super().__init__()

    self.dense1 = Dense(
        1200, 48, (1, 2), dtensor.Layout([input_mesh_dim, inner_mesh_dim1], mesh),
        activation=tf.nn.relu)
    self.bn = BatchNorm()
    self.dense2 = Dense(48, 2, (3, 4), dtensor.Layout([inner_mesh_dim1, output_mesh_dim], mesh))


  def __call__(self, x):
    y = x
    y = self.dense1(y)
    y = self.bn(y)
    y = self.dense2(y)
    return y

WORLD = dtensor.create_mesh([("world", 8)], devices=DEVICES)

model = MLP([dtensor.Layout.replicated(WORLD, rank=2),
             dtensor.Layout.replicated(WORLD, rank=2)])
```

PyTorch DTensor

- Newer implementation of distributed tensors in PyTorch
  - Follows a similar path to TensorFlow dtensor
- Two components
  - DistributedTensor API
  - module level API for nn.Module with “distributed” parameters.
- Follows similar approaches to TensorFlow mesh and layouts

```
mesh = init_device_mesh("cuda", (int(os.environ["WORLD_SIZE"]),))
example_tensor = torch.randn((653, 10))
dtensor = distribute_tensor(example_tensor, mesh, [Shard(0)])
print(f"on rank: {dist.get_rank()}, dtensor global shape: {dtensor.shape}, local shape:
{dtensor.to_local().shape}")
on rank: 2, dtensor global shape: torch.Size([653, 10]), local shape: torch.Size([164, 10])
on rank: 1, dtensor global shape: torch.Size([653, 10]), local shape: torch.Size([164, 10])
on rank: 0, dtensor global shape: torch.Size([653, 10]), local shape: torch.Size([164, 10])
on rank: 3, dtensor global shape: torch.Size([653, 10]), local shape: torch.Size([161, 10])
```

PyTorch DTensor

- Can use DistributedTensor in Model/Module classes

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.distributed import DeviceMesh
from torch.distributed.distributed_tensor import DTensor
import torch.distributed as dist
class DistributedLinear(nn.Module):
    def __init__(self, in_features, out_features, device_mesh, spec):
        super(DistributedLinear, self).__init__()
        self.in_features = in_features
        self.out_features = out_features
        self.device_mesh = device_mesh
        self.spec = spec

        # Initialize weights and biases as DTensors
        self.weights = DTensor(torch.randn(out_features, in_features), device_mesh,
spec)
        self.bias = DTensor(torch.randn(out_features), device_mesh, spec)

    def forward(self, x):
        # Perform the linear operation using DTensor's matmul
        # Note: Adjust the operation to match DTensor's API and your distribution spec
        return F.linear(x, self.weights.to_local(), self.bias.to_local())
```

PyTorch DTensor

```
import torch.nn as nn
from torch.distributed._tensor import Shard, distribute_tensor, distribute_module, init_device_mesh

class MyModule(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(8, 8)
        self.fc2 = nn.Linear(8, 8)
        self.relu = nn.ReLU()

    def forward(self, input):
        return self.relu(self.fc1(input) + self.fc2(input))

mesh = init_device_mesh("cuda", (4,))

def shard_params(mod_name, mod, mesh):
    col_linear_placement = [Shard(0)]
    # shard fc1 and fc2
    if isinstance(mod, nn.Linear):
        for name, param in mod.named_parameters():
            dist_param = nn.Parameter(
                distribute_tensor(param, mesh, col_linear_placement)
            )
            mod.register_parameter(name, dist_param)

sharded_module = distribute_module(MyModule(), mesh, partition_fn=shard_params)
```

PyTorch DTensor

- Limits optimisation potentials to individual tensor operations
  - Other approaches, i.e FSDP or DDP can see the full graph and may be able to overlap comms/calc, fuse operations, etc...
- Requires low level structuring of individual layers
  - Same as TensorFlow approach
  - Good and bad points
- Really useful as a building block for higher level approaches
  - Unless you're significantly engineering your model

Summary

- Model parallelisation is a harder task to undertake
  - Required for very large models
- Different approaches
  - Distribute the tasks: Pipeline
  - Split the tasks: Tensor
- Support is available for pipeline parallelism in standard frameworks
  - Requires some manual placement/optimisation
  - Can be hard to load balance or ensure full occupation
- Tensor parallelism is more complicated
  - But automatic/easy options are being created/provided